

Knowledge graph design review

Five design decisions taken before any triples were written, then five views of the graph they produce over the 24 dataset tables. Read to agree the vocabulary, not the code.

LAYERS	REUSED	SNAPSHOTS	CITATIONS
Inputs · Runs · Overrides	PROV-O · SOSA · OWL-Time · SKOS · SHACL · QUDT · DCAT/VoID	Referral + Type-2 states	Score node + cites edges

Five design decisions

Each one changes what triples exist, so each was taken before the loader was written. Two slides each: what the data actually does and what the alternatives were, then the choice and its reasoning. The five views afterwards are the consequence, not the question.

Nº	THE QUESTION	WHAT WAS CHOSEN
1	A row per referral per day — what becomes a node?	Referral + a state per material change, valid from one date to the next; wait counters derived, not stored
2	How does a broken rule become a fact?	SHACL, adopted now: structural shapes with the mappings, <code>sh:sparql</code> rule constraints after the first ranking
3	Where do units on the vitals live?	On the measurement type, not the reading — six <code>qudt:hasUnit</code> triples
4	What represents a patient with no national identifier?	Nothing. No Person node, no placeholder, no blank node — the absence stays a query result
5	Does the generated rationale text go in the graph?	Yes — its own write-once named graph that no agent reads, typed in PROV-O

Why one referral appears in the data 200 times

~110,000 rows in `referral_daily`
describe roughly 6,000 referrals.

Every morning each hospital exports its entire waiting list, and every morning's export is kept. A referral that has been waiting 200 days therefore appears 200 times — once per file — with her details as they stood that day.

On most of those days nothing about her changed. The same three lines below repeat for weeks at a time; only the wait number moves, and it moves by exactly one.

PW-0412	18 Aug	awaiting_triage	wait 31
PW-0412	25 Aug	triaged	wait 38
PW-0412	26 Aug	triaged	wait 39
PW-0412	27 Aug	triaged	wait 40
PW-0412	28 Aug	triaged	wait 41

Two things changed in six weeks: triage came back on the 22nd, and the hospital cancelled her twice. Everything else is repetition produced by how the file is made.

WHY IT CANNOT SIMPLY BE FLATTENED

Keeping only the newest row would be smallest of all, and would destroy the product. The whole claim of this system is that a ranking can be reconstructed: *what did it see on the 9th, and why did it place her second?* That question needs the past, so history has to survive in some form.

SO THE CHOICE IS BETWEEN TWO SHAPES

A · A node per day

A daily photograph. Any past day is a direct lookup, and the loader is trivial — but the graph stores the same unchanged facts 200 times: ~1.2M state triples, and about 1.6M in total.

B · A node per change

A change log. Each node records the period a set of facts held for, from one date until the next change. No duplicates — but "what did it see on the 9th" becomes a range question rather than a lookup.

One referral, a state per material change

THE QUESTION

The waiting list is re-read every morning and each morning is kept, so a referral waiting 200 days appears 200 times. Does the graph mint a node per day, or per change?

DECIDED

Never-changing fields sit on **eat:Referral**. An **eat:ReferralState** is minted only when a field materially changes, carrying `validFrom` and `validTo`. The four wait counters are **derived at query time**, not stored.

WHY

The repetition is an artefact of how the file is produced, not a fact about the patient. A state is a claim about a *period*, which is what Type 2 history models — and this is the layer where the team has the authority to choose it. Deriving the counters from one shared SPARQL fragment means the agents and the rule checker cannot disagree about a wait, which was the original reason for storing them.

```
eatd:referral-state/9001/PW-0412/2026-08-22
```

```
a eat:ReferralState ;
eat:stateOf eatd:referral/9001/PW-0412 ;
eat:validFrom "2026-08-22"^^xsd:date ;
eat:validTo "2026-08-25"^^xsd:date ;
eat:triageStatus eat:Triaged .
```

```
# nothing on the 23rd or 24th: nothing changed
# adjusted wait = validFrom → as-of, minus the
# suspension intervals. Derived, never stored.
```

WHAT IT BUYS

~500k triples instead of ~1.6M, and no duplicated arithmetic anywhere in the system.

WHAT IT COSTS

"What did the system see on the 9th" becomes a range query, so a citation must name the **state IRI**, never the date alone.

Guard. Type 2 only holds if states never overlap — enforced by a `sh:sparql` shape, not by trust.

A rule written in a document cannot be broken loudly

The rules already exist as data — the 28-day urgent timeframe, the 13-week semi-urgent one, and the hard constraint that no referral may ever be ranked above one of higher clinical priority. What does not exist is anything that **checks a ranking against them and says so**.

Written in Python only, a violation surfaces as a failed test in a build log. The clinician looking at the list sees nothing, and the reason a row is wrong is not attached to the row.

TWO VERY DIFFERENT KINDS OF WRONG

day 41 of 28

The hospital did not meet a national target. True, worth reporting, and not something the software did. It should be **recorded**, with its detail.

semi-urgent above urgent

Our own ranking crossed a category boundary. That is a defect in what we produced, and it should **stop the build**.

Today both are invisible in exactly the same way, which is the real problem.

WHY THE ONTOLOGY ALONE WILL NOT DO IT

OWL — the language the ontology is written in — reasons under the **open-world assumption**. It adds what must be true from what you have said, and it never objects to what you have not. Declare that urgent outranks semi-urgent, then produce a ranking that ignores it, and OWL reports nothing: no statement said positions must respect that order, so there is no contradiction to find.

This is not a gap in OWL. Inference and validation are different jobs, and only the second one is what a rule check needs.

A · Keep the rules in Python

Nothing new to learn. But the rulebook lives in two places, and the tests, the interface and the ontology can drift apart without anyone noticing.

B · SHACL shapes

A constraint language for RDF: shapes are data, they run against the store, and one file can drive both the build check and the warning a clinician sees.

SHACL now, so a rule can be broken visibly

THE QUESTION

The dataset already stores its rules as data. What turns one of them being broken into something the system reports rather than something a reader has to notice?

DECIDED

Adopt SHACL from the start. Structural shapes are written **alongside the mappings** so they catch loader errors while the mapping is still fresh; the `sh:sparql` rule constraints are added **once the coordinating agent produces its first ranking**, because they need a placement to target.

WHY NOT OWL ALONE

OWL is open-world: given a referral ranked above a higher-priority one, it infers, it never complains. `RULE-ORDER` is a constraint, not an entailment. One shape file then serves both the CI gate and the clinician's rule warning, so the test and the interface can never disagree about what was violated.

WHICH RULES FAIL THE BUILD

RULE-ORDER **Shape.** Ranking across a category boundary is a defect in our own output — the build fails.

RULE-TIEBREAK **Shape.** Same reasoning: ordering we produced ourselves.

RULE-CRT-* **Rule check.** A 41-day urgent wait is a fact about the hospital, not an invalid graph. Recorded as `passed false` with its detail.

TURNAROUND **Rule check.** Same: describes the service, not the software.

Both outputs, one source. A violation fails CI *and* materialises as an `eat:RuleCheck` in the run graph. A clinician may still override a rule warning knowingly, so the shapes target `eat:Decision` in the run graph and never the override graph.

The graph records 38.2 and never says of what

~12,000 rows in observations, six of them measured values.

One row of observations for PW-0412 reads like this. A nurse reads it without hesitation. A query cannot: nothing in the data says which number is in beats per minute, which is in degrees Celsius, and which is a percentage.

```
hr 104   rr 22   temp 38.2   spo2 96
pain 7    avpu A   news2 5    mts orange
```

SOSA — the observation vocabulary this project reuses — carries a result and a time, but has no opinion about units. So the unit has to be stated somewhere, or the graph is only readable by people who already know the convention.

It matters beyond tidiness. The moment this data meets another system, an unlabelled 38.2 is a number that could be Celsius or Fahrenheit, and a capacity or urgency rule built on it is unsafe to move.

THERE ARE ONLY TWO PLACES A UNIT CAN LIVE

A · On every reading

Each measurement states its own unit. Survives mixed data of any kind — and costs one extra statement per value: six measured columns across 12,000 observations is roughly **72,000 statements** that all say the same six things.

B · On the thing being measured

Heart rate is *always* in beats per minute, so say it once about heart rate itself: **six statements** in the ontology file, and every reading inherits them.

B is only true while a column never carries mixed units. That assumption is safe in this dataset because it is generated — one unit per column by construction — and it is exactly the assumption that fails in real hospital data.

The project has already seen it fail: cleaning the real MIMIC source turned up a Fahrenheit reading sitting in the Celsius column, alongside a diastolic pressure of 879.

A unit belongs to the measurement type

Affects: observations — ~12,000 rows,
six measured columns.

THE QUESTION

SOSA gives a result but says nothing about units. A temperature of 38.2 and an oxygen saturation of 96 are ambiguous without them. Where does the unit go?

DECIDED

`qudt:hasUnit` on the **observable property**, declared once in the ontology. Not on the individual observation.

WHY

A unit is a property of what is being measured, not of each measurement, and this dataset has exactly one unit per column by construction. Six triples in the ontology file replace six per observation across twelve thousand of them — cheaper than the option that was supposed to be cheapest, and semantically the more accurate claim.

Limitation, on the record. Unit-on-property assumes one unit per measurement type forever. The project's own MIMIC cleaning found a Fahrenheit reading sitting in the Celsius column: real ingestion breaks this assumption, and at that point units must move onto the observation.

```
eat:heartRate      qudt:hasUnit unit:PER-MIN .
eat:respiratoryRate qudt:hasUnit unit:PER-MIN .
eat:systolicBP     qudt:hasUnit unit:MilliM_HG .
eat:diastolicBP    qudt:hasUnit unit:MilliM_HG .
eat:temperature    qudt:hasUnit unit:DEG_C .
eat:oxygenSaturation qudt:hasUnit unit:PERCENT .
```

```
# and the reading carries no unit of its own:
eatd:obs/9001/PW-0412/2026-08-19T09:14:00/hr
  a sosa:Observation ;
  sosa:observedProperty eat:heartRate ;
  sosa:hasSimpleResult 104 .
```

Pain, chief complaint, AVPU, NEWS2 and the MTS colour are unitless by nature — they stay as they are. The distinction between measured, reported and judged is already carried by the three `sosa:ObservableProperty` subclasses.

Two records, one woman, and often no way to tell

About a third of patient records carry no national identifier — deliberately.

A hospital's patient number is unique *inside that hospital only*. The same woman attending two hospitals has two unrelated numbers. The one thing that can link them is the national health identifier, and it is frequently missing.

9001	MRN-004412	IHI-7781004	F	1959-03-11	
9002	MRN-119003	IHI-7781004	F	1959-03-11	← same person
9001	MRN-005120	(none)	M	1997-06-02	← unknowable

The first two rows are one person, and we know it. The third man may be waiting at three other hospitals; as far as this data can tell, he is a stranger everywhere.

This is product-relevant, not bookkeeping. "Also waiting in dermatology here and cardiology at Kilbrannan" is evidence a ranking can cite. With no identifier, that sentence cannot honestly be written.

THE TEMPTING FIX, AND THE TWO WAYS TO GO

A · Give everyone a person node

Mint one anyway when the identifier is absent, so every patient has the same shape and no query needs a special case. Uniform, convenient — and it asserts a national identity that does not exist. Two such nodes can never be compared with each other, so the node has the appearance of identity and none of its use.

B · Leave the gap open

No identifier, no person node at all. Queries must then handle a missing link explicitly — and the missing link stays countable, which is the finding the dataset exists to show.

The dataset documentation is explicit that the gap is the point: it is what a national shared care record would fix. A design that papers over it deletes the argument.

No identifier, no Person node

THE QUESTION

Roughly a third of patients have no national health identifier, and that gap is deliberate — it is what a shared care record would fix. What does the graph put there?

DECIDED

Nothing at all. No **eat:Person** node, no placeholder IRI, no blank node. Two classes rather than one: the hospital-scoped **eat:Patient** record always exists, the national **eat:Person** only where an identifier does, and `eat:isRecordOf` — functional — joins them where both are present.

WHY

A placeholder would assert that a person exists while being uncitable and uncomparable: the appearance of national identity with none of its substance. Under the open-world assumption OWL concludes nothing about the missing link, which is the honest position — we do not know whether this patient is waiting elsewhere. `FILTER NOT EXISTS` is negation as failure over what is asserted, which is exactly the claim the data supports: *we cannot link this patient*. The absence is a query result, not an axiom.

```
eat:Patient a owl:Class .    # always exists
eat:Person  a owl:Class .    # only with an IHI
```

```
eat:isRecordOf a owl:ObjectProperty,
               owl:FunctionalProperty ;
rdfs:domain eat:Patient ;
rdfs:range  eat:Person .
```

```
SELECT (COUNT(?p) AS ?unlinkable) WHERE {
  ?p a eat:Patient .
  FILTER NOT EXISTS { ?p eat:isRecordOf ?person }
}
```

Limitation, on the record. In the published data 3,412 patients carry an identifier and none occurs at more than one hospital, so every Person node has exactly one Patient pointing at it. The structure is right; the data does not yet exercise it. PW-DEM0-06 and 07 have the shape but do not demonstrate multi-list visibility, and `cross_hospital_share` is read by nothing. No demo should rest on cross-hospital linkage without checking the loaded profile first.

The one sentence a clinician actually reads

Feature 7 promises the model cannot introduce unsupported facts.

Each ranked row carries one line of explanation, written by a language model from evidence the graph has already cited:

"Urgent, day 41 of 28. Early warning score 5 recorded 19 Aug. Cancelled twice by the hospital. No surgical clinic slot before 2 Sept."

Every clause there has to come from something stored. The promise is that the model may not change a score, change a rank, or add a fact of its own — and that promise is worth nothing unless it can be **checked after the fact**.

Checking it needs two things a plain text field does not keep: the exact set of citations that were in the prompt when that sentence was produced, and which model version produced it.

AND THE RISK OF STORING IT WELL

Put the sentence in the graph beside the vitals and the bed counts, and it starts to look like evidence. If any agent ever reads it back, generated prose becomes an input to the next decision — the exact failure this architecture is built to prevent.

A · A text column beside the ranking

Simple, and safely outside the graph. It stores the string and loses the link: the citation set is a *set of identifiers*, which is painful as a column and natural as edges.

B · Nodes in the graph

The sentence, what it used, who produced it and when, all as one auditable structure — provided something guarantees no agent ever reads it.

In the graph, in its own graph, write-once

THE QUESTION

Storing the generated sentence makes it auditable, but it also puts model output in the same store as clinical evidence. In the graph, or beside it in a column?

DECIDED

In the graph, in `.../graph/rationale`, write-once, with an invariant nobody breaks: **no agent's SPARQL reads from it**. The same discipline as `ground_truth`, for a different reason — ground truth is withheld to prevent circularity, rationale is isolated so generated prose is never mistaken for evidence.

WHY NOT A COLUMN

A column stores the string but loses the link. To audit a sentence you need to know which citations were in the prompt that produced it, and that is a set of IRIs — natural as graph structure, painful as a foreign key. The IRI is keyed on hospital, date and pathway, so a referral re-ranked the next day gets a new rationale and the old one stays addressable. Never overwritten, the same append-only discipline the output tables already follow.

```
eatd:rationale/9001/2026-08-28/PW-0412
```

```
  a eat:Rationale ;
  eat:rationaleText "...";
  prov:wasGeneratedBy eatd:activity/rationale-gen/... ;
  prov:used eatd:citation/... , eatd:citation/... ;
  prov:wasAttributedTo eatd:agent/claude-opus-5 ;
  eat:promptHash "sha256:..." .
```

PROV-O has no dedicated text property, so rather than overload `prov:value` the deck uses `eat:rationaleText` with an `rdfs:comment` recording the choice. Either is defensible; apply one consistently.

The prov:used set is the point. Every entity named in the sentence must appear in the citation set it used — a containment check that runs in CI, which is how the promise gets demonstrated rather than asserted. The prompt hash and model version tell you which rationales predate a model change without re-reading them.

The vocabulary stack

CONFIRMED FOR THIS BUILD

```
@prefix eat: <https://nonsocynthia.github.io/Tus-Aite-
TechIreland-Challenge/kg/ns#> .
@prefix eatd: <https://nonsocynthia.github.io/Tus-Aite-
TechIreland-Challenge/kg/id/> .
@prefix prov: <http://www.w3.org/ns/prov#> .
@prefix sosa: <http://www.w3.org/ns/sosa/> .
@prefix time: <http://www.w3.org/2006/time#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix dcat: <http://www.w3.org/ns/dcat#> .
@prefix void: <http://rdfs.org/ns/void#> .
@prefix sct: <http://snomed.info/id/> .
@prefix sh: <http://www.w3.org/ns/shacl#> .
@prefix qudt: <http://qudt.org/schema/qudt/> .
@prefix unit: <http://qudt.org/vocab/unit/> .
```

Every IRI is minted from the composite keys the dataset already uses, so the same row always produces the same node: `eatd:referral/9001/PW-0412`. No blank nodes anywhere in the input layer — they cannot be cited. Slash-bearing names are shown abbreviated throughout this deck; in the file they are written as full `<...>` IRIs, since Oxigraph rejects an unescaped `/` in a prefixed name.

ADDED BY DECISION

SHACL (sh:) — adopted

OWL is open-world: it will infer, not complain. Structural shapes are written alongside the mappings; the `sh:sparql` rule constraints for the five `ref_rules` land once the coordinator produces its first ranking.

QUDT — on the property, not the reading

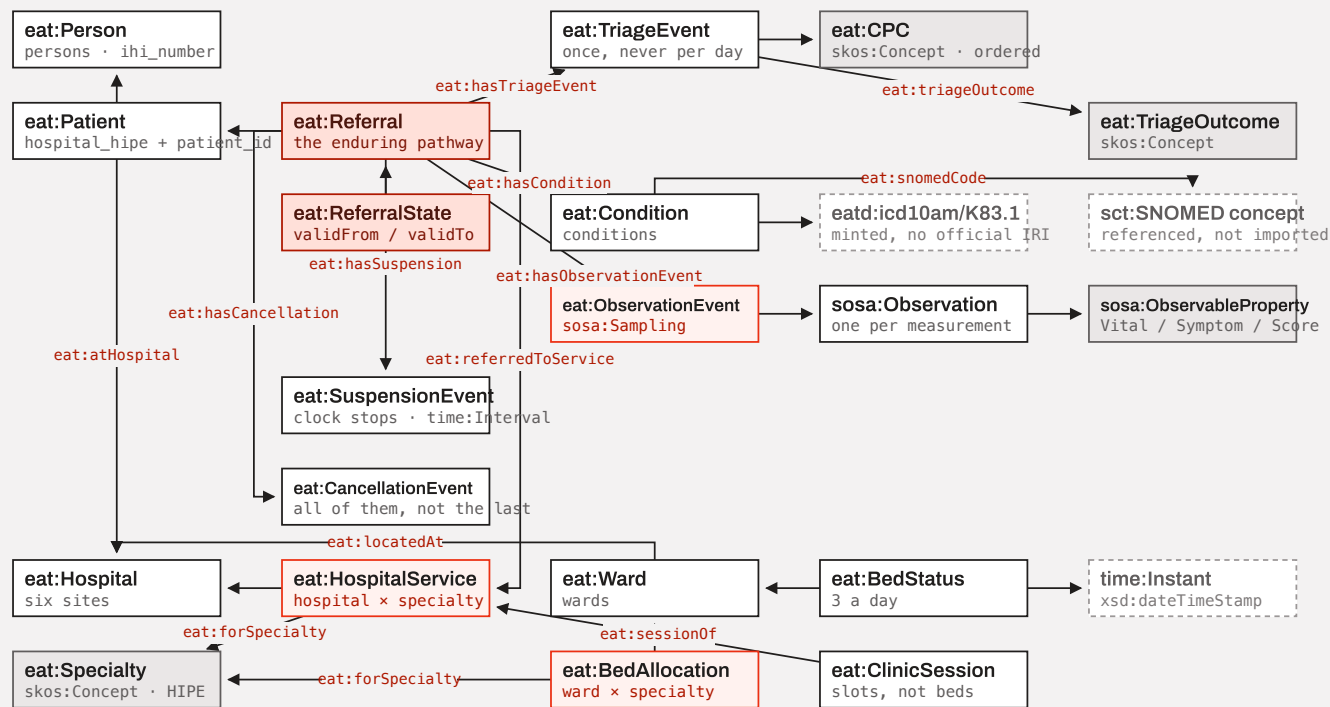
A unit belongs to the measurement type, and this dataset has one unit per column by construction. Six `qudt:hasUnit` triples in the ontology, not six per observation across 12,000 of them.

DELIBERATELY NOT USED

FHIR RDF — no interop requirement in this challenge, and it would double every class. Keep the field names MDS-aligned so the mapping stays possible later. **The SNOMED CT ontology** — licensed and enormous; reference `sct`: IRIs, import nothing. **FOAF/org**: — no people or organisations are named in this dataset by design.

The data layer — what the system sees

referral spine
 n-ary node
 skos:Concept
 external IRI



READ THIS WAY

eat:HospitalService is the join the graph must keep: cardiology at 9001 is not cardiology at 9002. Every bed and slot question hangs off it.

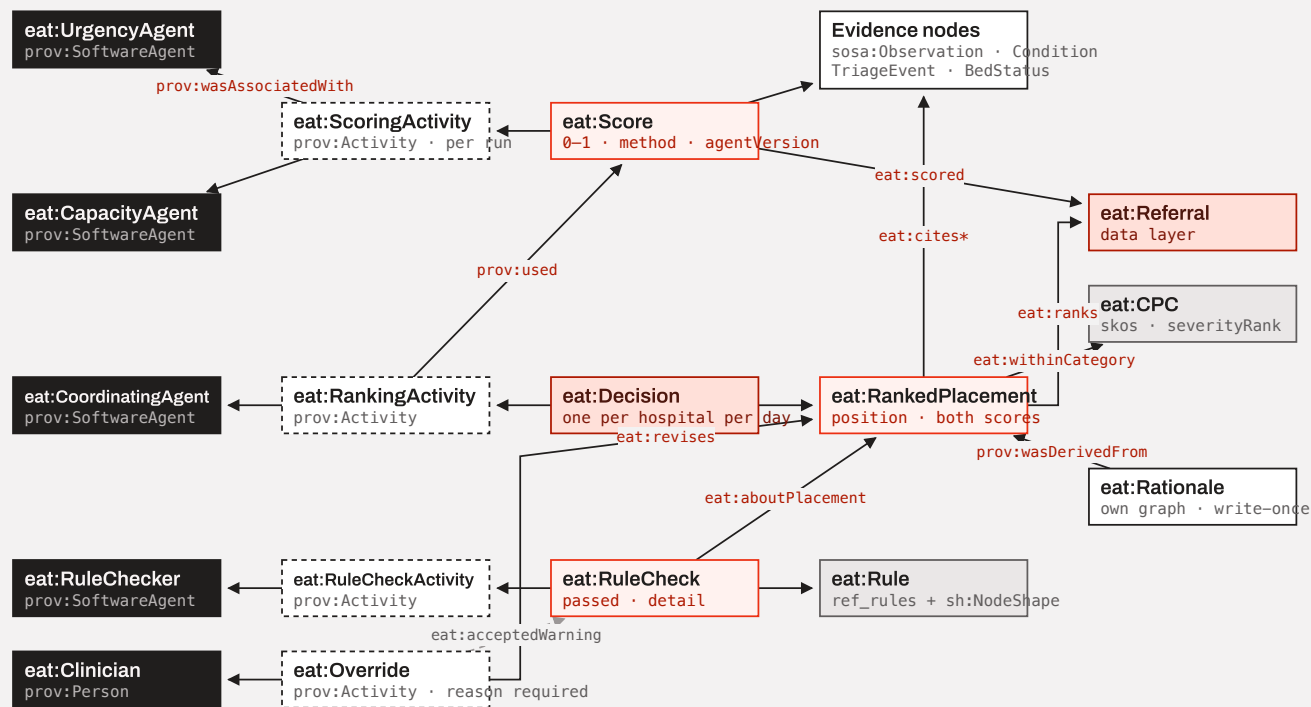
eat:BedAllocation reifies **ward_specialty** so **nominal_beds** has somewhere to live — it makes contention between specialties queryable.

sosa:Observation is one node per measurement, not per chart row, so a score can cite the exact heart rate it used. The three kinds from §7.6 become subclasses of **sosa:ObservableProperty**.

Not drawn:
sosa:hasFeatureOfInterest to the Patient, and
ground_truth — no triples, any graph.

Data-layer properties

PROPERTY	DOMAIN	RANGE	CARD.	WHY / SOURCE
eat:isRecordOf	eat:Patient	eat:Person	0..1 (functional)	no IHI → no Person node, no placeholder
eat:atHospital	eat:Patient, eat:Referral	eat:Hospital	1	hospital_hipe
eat:forPatient	eat:Referral	eat:Patient	1	referral_daily.patient_id
eat:referredToService	eat:Referral	eat:HospitalService	1	hospital_hipe + specialty_hipe
eat:stateOf	eat:ReferralState	eat:Referral	1 (functional)	Type 2: a state per material change
eat:validFrom / eat:validTo	eat:ReferralState	xsd:date	1 / 0..1	open-ended until the next change
eat:hasTriageEvent	eat:Referral	eat:TriageEvent	0..1 (functional)	null until returned from triage (§10.3)
eat:assignedCategory	eat:TriageEvent	eat:CPC	0..1	triage_events.triage_category
eat:outranks	eat:CPC	eat:CPC	transitive	severity_rank — never code_value
eat:hasCondition	eat:Referral	eat:Condition	1..n	conditions, is_primary flags one
eat:icd10amCode	eat:Condition	IRI (minted)	1	ICD-10-AM has no public IRI scheme
eat:snomedCode	eat:Condition	sct: IRI	0..1	reference only, ontology not imported
sosa:hasMember	eat:ObservationEvent	sosa:Observation	1..n	one per non-null observations column
sosa:observedProperty	sosa:Observation	sosa:ObservableProperty	1	hr, spo2, pain, news2, mts_category ...
sosa:hasFeatureOfInterest	sosa:Observation	eat:Patient	1	the patient, not the referral
eat:providedBy	eat:HospitalService	eat:Hospital	1	the unit that owns a waiting list
eat:forSpecialty	HospitalService, BedAllocation	eat:Specialty	1	both of the schema-forced joins
eat:inWard	eat:BedAllocation	eat:Ward	1	ward_specialty + nominal_beds
eat:statusOf	eat:BedStatus	eat:Ward	1	three snapshots a day
eat:sessionOf	eat:ClinicSession	eat:HospitalService	1	the real outpatient constraint
eat:hasSuspension	eat:Referral	eat:SuspensionEvent	0..n	first-class, with its own interval
eat:hasCancellation	eat:Referral	eat:CancellationEvent	0..n	all of them, not just the latest



THE AUDIT TRAIL

A score is never a bare number:
it is an entity with
`prov:wasGeneratedBy` an
activity, a versioned agent, and
one `eat:cites` edge per
evidence row.

The role column of `decision_citations` becomes four sub-properties, not a reified statement:

```
eat:citesUrgencyEvidence
eat:citesCapacityEvidence
eat:citesTimeframeEvidence
eat:citesMultiListEvidence
⊆ eat:cites ⊆ prov:used
```

One query gets the whole explanation; the same query with a sub-property gets one section of the clinician's expander.

Decision-layer properties

PROPERTY	DOMAIN	RANGE	CARD.	WHY / SOURCE
eat:scored	eat:Score	eat:Referral	1	agent_scores
eat:scoreValue	eat:Score	xsd:decimal 0–1	1	never stored without method + version
eat:method / eat:agentVersion	eat:Score	xsd:string	1/1	reproducibility, not decoration
eat:cites	Score, RankedPlacement	evidence node	1..n	rdfs:subPropertyOf prov:used
eat:citesUrgencyEvidence	⊆ eat:cites	evidence node	0..n	decision_citations.role = urgency
eat:citesCapacityEvidence	⊆ eat:cites	evidence node	0..n	role = capacity
eat:citesTimeframeEvidence	⊆ eat:cites	eat:ReferralState, eat:Rule	0..n	role = timeframe
eat:citesMultiListEvidence	⊆ eat:cites	eat:Referral	0..n	role = multi_list (needs an IHI)
eat:hasPlacement	eat:Decision	eat:RankedPlacement	1..n	decision_rankings
eat:position	eat:RankedPlacement	xsd:positiveInteger	1	unique per decision + category
eat:ranks	eat:RankedPlacement	eat:Referral	1	the referral being placed
eat:withinCategory	eat:RankedPlacement	eat:CPC	0..1	stored so RULE-ORDER is checkable at a glance
eat:testedRule	eat:RuleCheck	eat:Rule	1	rule_checks — pass and fail both stored
eat:passed	eat:RuleCheck	xsd:boolean	1	a violation becomes a fact, not a log line
eat:revises	eat:Override	eat:RankedPlacement	1	from_position → to_position
eat:reason	eat:Override	xsd:string	1	no override without one
eat:acceptedWarning	eat:Override	eat:RuleCheck	0..n	warning and the decision to proceed, both kept
prov:wasGeneratedBy	Score, Decision, RuleCheck	prov:Activity	1	every written node has one
prov:wasAssociatedWith	prov:Activity	prov:Agent	1	agent or clinician, never anonymous
prov:wasDerivedFrom	eat:Rationale	eat:RankedPlacement	1	text may cite nothing the placement did not
eat:rationaleText	eat:Rationale	xsd:string	1	chosen over overloading prov:value
prov:used	eat:Rationale	citation set	1..n	exactly what was in the prompt
prov:wasAttributedTo	eat:Rationale	prov:SoftwareAgent	1	the model, by version string
eat:promptHash	eat:Rationale	xsd:string	1	sha256 — which rationales predate a change

TBox excerpt — the parts that carry a rule

```
eat:Referral a owl:Class ;
  rdfs:comment "The enduring pathway. Never per-day." .

eat:ReferralState a owl:Class ;
  rdfs:subClassOf [ a owl:Restriction ;
    owl:onProperty eat:stateOf ; owl:cardinality 1 ] .

eat:stateOf a owl:ObjectProperty, owl:FunctionalProperty ;
  rdfs:domain eat:ReferralState ; rdfs:range eat:Referral .

eat:hasTriageEvent a owl:ObjectProperty,
  owl:FunctionalProperty ;          # 0..1 – §10.3
  rdfs:domain eat:Referral ; rdfs:range eat:TriageEvent ;
  owl:inverseOf eat:triageEventOf .

# The three kinds of observation (§7.6)
eat:MeasuredVital      rdfs:subClassOf sosa:ObservableProperty .
eat:ReportedSymptom    rdfs:subClassOf sosa:ObservableProperty .
eat:ClinicalAssessment rdfs:subClassOf sosa:ObservableProperty .

# A unit is a property of the measurement type.
# Six triples here, not six × 12,000 readings.
eat:heartRate          qudt:hasUnit unit:PER-MIN .
eat:respiratoryRate    qudt:hasUnit unit:PER-MIN .
eat:systolicBP         qudt:hasUnit unit:MilliM_HG .
eat:diastolicBP        qudt:hasUnit unit:MilliM_HG .
eat:temperature        qudt:hasUnit unit:DEG_C .
eat:oxygenSaturation   qudt:hasUnit unit:PERCENT .
```

```
# Severity order lives in the graph, not in code.
eat:CPCScheme a skos:ConceptScheme ;
  skos:prefLabel "Clinical Prioritisation Category" .

eatd:cpc/1 a skos:Concept ;          # Urgent
  skos:inScheme eat:CPCScheme ;
  skos:notation "1" ;
  eat:severityRank 1 ;
  eat:crtDays 28 .

eatd:cpc/3 a skos:Concept ;          # Semi-Urgent
  eat:severityRank 2 ; eat:crtDays 91 .

eatd:cpc/2 a skos:Concept ;          # Routine
  eat:severityRank 3 .               # no crtDays

eat:CPCOrder a skos:OrderedCollection ;
  skos:memberList ( eatd:cpc/1 eatd:cpc/3 eatd:cpc/2 ) .

eat:outranks a owl:ObjectProperty, owl:TransitiveProperty ;
  rdfs:domain eat:CPC ; rdfs:range eat:CPC .
eatd:cpc/1 eat:outranks eatd:cpc/3, eatd:cpc/2 .
eatd:cpc/3 eat:outranks eatd:cpc/2 .
```

Shapes now, `sh:sparql` after the first ranking

WRITTEN WITH THE MAPPINGS — STRUCTURAL

```
eat:ReferralStateShape a sh:NodeShape ;
  sh:targetClass eat:ReferralState ;
  sh:closed true ;                # catches loader typos
  sh:ignoredProperties ( rdf:type ) ;
  sh:property [ sh:path eat:stateOf ;
    sh:class eat:Referral ;
    sh:minCount 1 ; sh:maxCount 1 ] ;
  sh:property [ sh:path eat:validFrom ;
    sh:datatype xsd:date ;
    sh:minCount 1 ; sh:maxCount 1 ] ;
  sh:property [ sh:path eat:validTo ;
    sh:datatype xsd:date ; sh:maxCount 1 ] .
```

Type 2 only works if the states never overlap.

```
eat:StateIntervalShape a sh:NodeShape ;
  sh:targetClass eat:Referral ;
  sh:sparql [ sh:message "overlapping states" ;
    sh:select ""SELECT $this WHERE {
      ?a eat:stateOf $this ; eat:validFrom ?af .
      ?b eat:stateOf $this ; eat:validFrom ?bf .
      FILTER(?a != ?b && ?bf > ?af)
      OPTIONAL { ?a eat:validTo ?at }
      FILTER(!BOUND(?at) || ?bf < ?at) }"" ] .
```

AFTER THE COORDINATOR RANKS — RULE CONSTRAINTS

```
# RULE-ORDER. A defect, not a judgement call.
eat:RuleOrderShape a sh:NodeShape ;
  sh:targetClass eat:Decision ;
  sh:sparql [
    sh:message "RULE-ORDER: {?lo} ranked above {?hi}" ;
    sh:select ""SELECT $this ?hi ?lo WHERE {
      $this eat:hasPlacement ?hi, ?lo .
      ?hi eat:position ?ph ; eat:withinCategory ?ch .
      ?lo eat:position ?pl ; eat:withinCategory ?cl .
      ?ch eat:outranks ?cl .
      FILTER(?pl < ?ph) }"" ] .
```

RULE-CRT stays a rule check, not a shape. A 41-day urgent wait is a fact about the hospital, not an invalid graph — the checker records passed `false` with its detail. Only `RULE-ORDER` and `RULE-TIEBREAK` describe something the system itself did wrong, so only those two fail the build.

A clinician override may break `RULE-ORDER` knowingly. The shape therefore targets `eat:Decision` in the run graph, never the override graph.

All 24 tables, and what each becomes

17 loaded in, 7 written out, 1 held out. Nothing is dropped, but five resolve to properties or concepts rather than classes.

TABLE	GRP	BECOMES	VERDICT	TABLE	GRP	BECOMES	VERDICT
referral_daily	A	eat:Referral + eat:ReferralState	split: stable vs per change	suspension_events	E	eat:SuspensionEvent	class + time:Interval
triage_events	A	eat:TriageEvent	class · once per referral	ref_specialty	F	skos:ConceptScheme	concepts, not a class
patients	B	eat:Patient	class · composite IRI	ref_codes	F	7 skos:ConceptSchemes	one scheme per code_table
persons	B	eat:Person	class · absent when no IHI	ref_rules	F	eat:Rule + sh:NodeShape	data and shape together
conditions	C	eat:Condition	class + external code IRIs	ground_truth	G	— nothing —	held out: no triples, any graph
observations	C	eat:ObservationEvent + sosa:Observation	one row → many nodes	agent_scores	out	eat:Score	entity + prov
hospitals	D	eat:Hospital	class	agent_citations	out	eat:cites edges	properties, not a class
hospital_specialty	D	eat:HospitalService	n-ary node	decisions	out	eat:Decision	entity + prov
wards	D	eat:Ward	class	decision_rankings	out	eat:RankedPlacement	n-ary node
ward_specialty	D	eat:BedAllocation	n-ary node	decision_citations	out	eat:cites sub-properties	role becomes the predicate
bed_status	D	eat:BedStatus	class · time-stamped	rule_checks	out	eat:RuleCheck	n-ary node
clinic_sessions	D	eat:ClinicSession	class · time-stamped	overrides	out	eat:Override	prov:Activity
cancellation_events	E	eat:CancellationEvent	class				

One row of referral_daily, emitted

```

GRAPH <.../kg/graph/inputs/2026-08-28> {

eatd:referral/9001/PW-0412 a eat:Referral ;
  eat:pathwayNumber "PW-0412" ;
  eat:atHospital      eatd:hospital/9001 ;
  eat:forPatient      eatd:patient/9001/MRN-004412 ;
  eat:referredToService eatd:service/9001/2600 ;
  eat:referralDate      "2026-07-15"^^xsd:date ;
  eat:referralReceivedDate "2026-07-18"^^xsd:date ;
  eat:gpPriority        eatd:gp-priority/1 ;
  eat:referralSource eatd:referral-source/12 ;
  eat:highClinicalOrSocialNeeds false ;
  eat:hasTriageEvent eatd:triage-event/TE-0412 ;
  eat:hasCondition    eatd:condition/9001/PW-0412/K83.1 ;
  eat:hasObservationEvent
    eatd:obs-event/9001/PW-0412/2026-08-19T09:14:00 ;
  eat:hasCancellation
    eatd:cancellation/9001/PW-0412/2026-08-11 ,
    eatd:cancellation/9001/PW-0412/2026-08-24 .

eatd:patient/9001/MRN-004412 a eat:Patient ;
  eat:isRecordOf eatd:person/IHI-7781004 .
# No IHI → no isRecordOf and no eat:Person node.
# Absence stays a query result, not a placeholder.
}

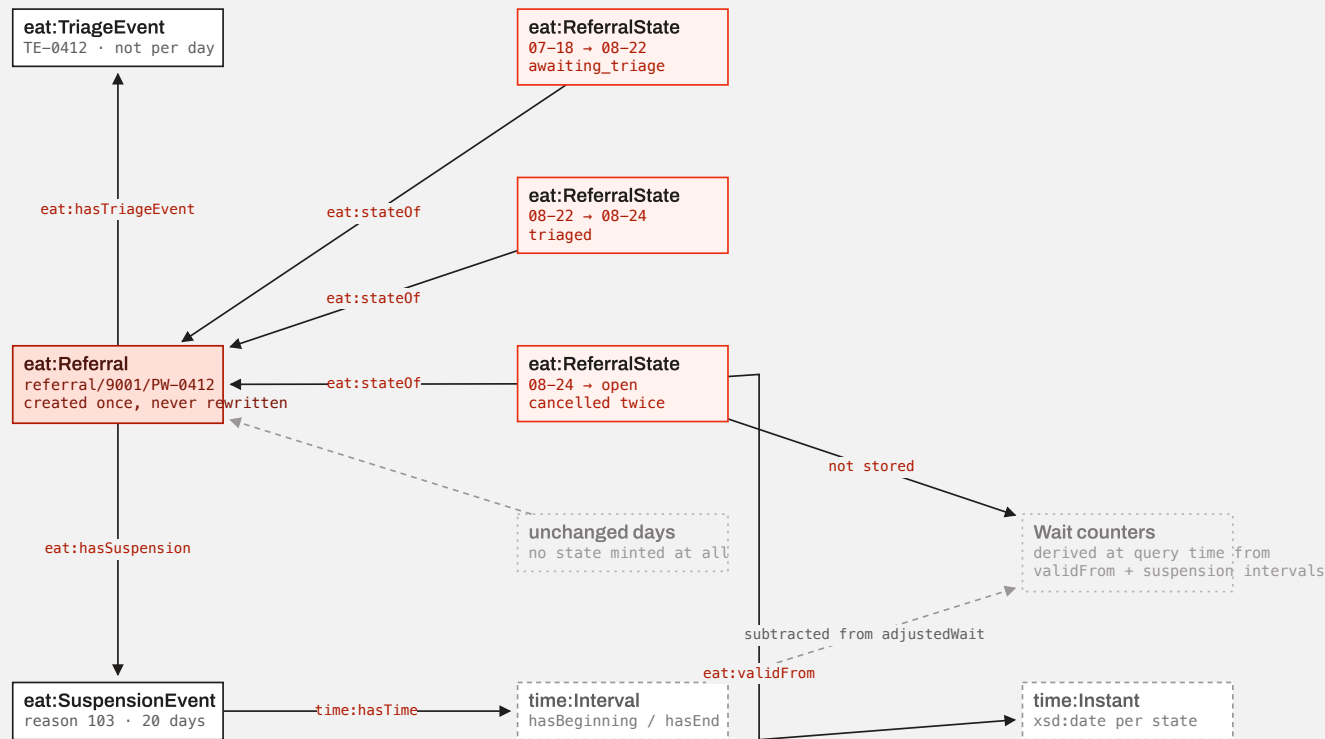
# Type 2: minted only on material change.
eatd:referral-state/9001/PW-0412/2026-08-22
  a eat:ReferralState ;
  eat:stateOf      eatd:referral/9001/PW-0412 ;
  eat:validFrom    "2026-08-22"^^xsd:date ;
  eat:validTo      "2026-08-25"^^xsd:date ;
  eat:triageStatus eat:Triaged .
# no state for the 23rd or 24th: nothing changed.
# the four wait counters are derived at query
# time from validFrom and the suspension intervals.

eatd:condition/9001/PW-0412/K83.1 a eat:Condition ;
  eat:icd10amCode eatd:icd10am/K83.1 ;
  eat:isPrimary true ;
  rdfs:label "Obstruction of bile duct" .

eatd:obs/9001/PW-0412/2026-08-19T09:14:00/hr
  a sofa:Observation ;
  sofa:observedProperty eat:heartRate ;    # MeasuredVital
  sofa:hasSimpleResult 104 ;              # unit from the property
  sofa:resultTime "2026-08-19T09:14:00"^^xsd:dateTime ;
  sofa:hasFeatureOfInterest
    eatd:patient/9001/MRN-004412 .

```

110,000 rows → one referral, a state per change



Decided. Never-changing fields sit on the Referral. A ReferralState is minted only on material change, valid from one date to the next.

COST

~6,000 referrals × ~14 triples = **84k**

~3–5 states each × ~7 triples ≈

170k

Everything else ≈ 250k. ~**500k**

triples against 1.6M for a state per day.

THE FOUR WAIT COUNTERS

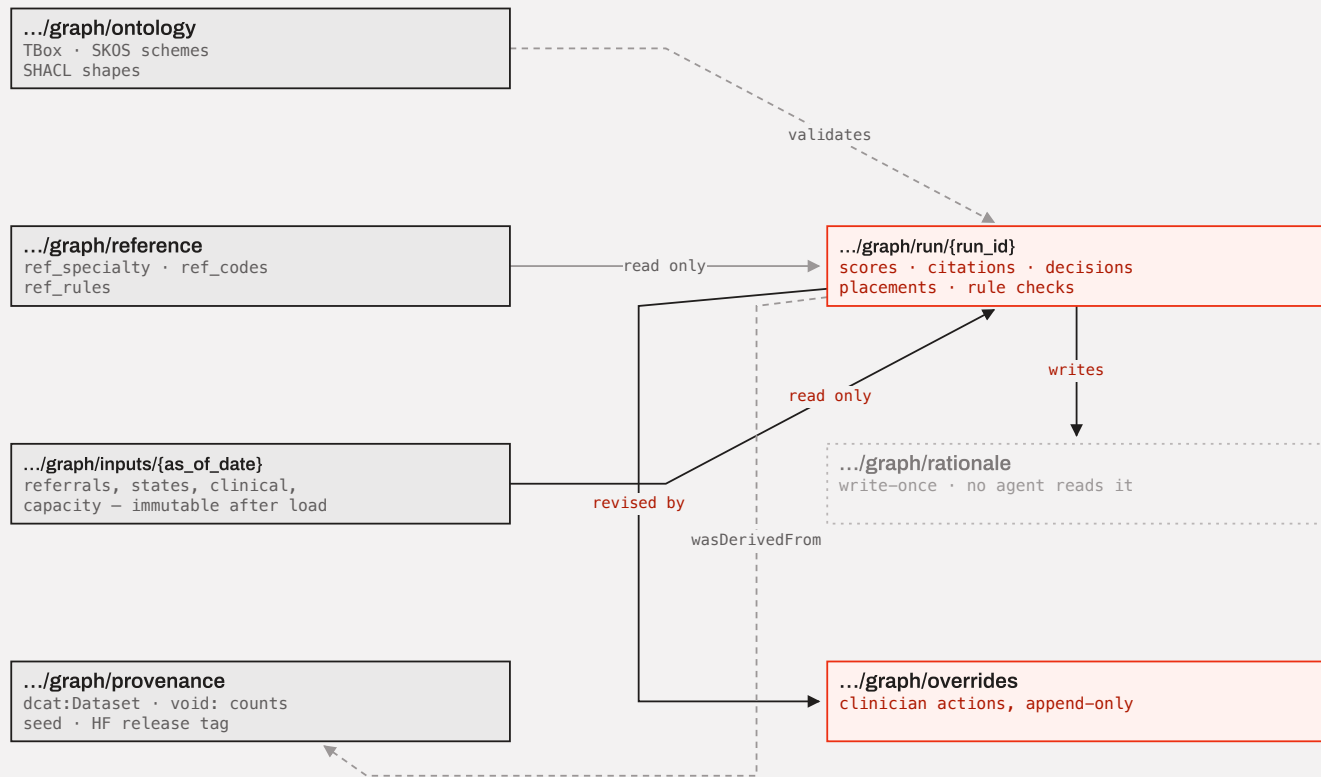
Derived at query time from validFrom and the suspension intervals, not stored. One SPARQL fragment, shared by the agents and the rule checker, so they cannot disagree.

WHAT REPLAY COSTS NOW

"What did the system see on the 9th" becomes a range query — validFrom ≤ 9th < validTo — rather than a lookup. A citation must therefore name the state IRI, never the date alone.

Seven layers, one write direction

Section 8's input/output separation becomes a named-graph rule, not a convention. Replay = drop one run graph and re-run.



ENFORCED, NOT ASSUMED

```

# must return false
ASK {
  GRAPH ?g { ?s ?p ?o }
  FILTER(STRSTARTS(STR(?g),
    ".../kg/graph/inputs/") )
  FILTER(?p IN (eat:hasScore,
    eat:cites, eat:position))
}
  
```

One CI test per run: no decision predicate may appear in an input graph, and no input triple may change between two runs of the same seed. The shapes run against the run graph, so a violation is reported without ever writing to the inputs.

Overrides get their own graph so human judgement is never mixed with machine output.

Rationales get theirs — write-once, read by no agent. Generated prose must never re-enter as evidence.

Provenance holds `dcat:Dataset`, the release tag and the seed.

Which agent may touch which class

R reads **W** writes · no access

CLASS	URGENCY AGENT	CAPACITY AGENT	COORDINATOR	RULE CHECKER	CLINICIAN UI
eat:Referral / ReferralState	R	R	R	R	R
eat:TriageEvent + eat:CPC	R	·	R	R	R
eat:Condition	R	·	·	·	R
sosa:Observation	R	·	·	·	R
eat:HospitalService	R	R	R	R	R
eat:BedStatus / BedAllocation	·	R	·	·	R
eat:ClinicSession	·	R	·	·	R
eat:Score	W	W	R	·	R
eat:Decision / RankedPlacement	·	·	W	R	R
eat:cites (all sub-properties)	W	W	W	R	R
eat:Rule	R	R	R	R	R
eat:RuleCheck	·	·	·	W	R
eat:Rationale	·	·	·	·	R
eat:Override	·	·	·	·	W
ground_truth	·	·	·	·	·